PFLICHTENHEFT

AO-Server Tool (DayZ Management Framework)

Datum: 30. Oktober 2026

Status: Freigegeben zur Umsetzung

1. Systemarchitektur und Laufzeitumgebung

- **Betriebssystem:** Linux-Umgebung (optimiert für Ubuntu 24.04 LTS).
- **Technologie-Stack:** Bash-Scripting für die Kernlogik, `tmux` für das Sitzungsmanagement im Hintergrund, `bercon-cli` als Wrapper für BattlEye-RCON-Verbindungen.
- **Verzeichnis- und Rechtestruktur:**
  - Anlegen eines dedizierten Systembenutzers (z. B. `dayz_ao`) zur strikten Rechte-Isolierung.
  - Verzeichnisstruktur im Home-Verzeichnis (`/home/dayz_ao/<servername>/`):
    - `/steamcmd/`: Beinhaltet die SteamCMD-Binaries.
    - `/server_data/`: Root-Verzeichnis des DayZ-Servers.
    - `/server_manager/`: Skripte, CLI-Dashboard und Konfigurationsdateien (`.cfg`) des AO-Tools.

2. CLI-Dashboard und Monitoring

- **Darstellung:** Ausgabe des hinterlegten ASCII-Art-Logos bei Aufruf der Hauptdatei.
- **Live-Status-Routing:** Implementierung einer Abfrageschleife über den Steam Query Port (via `qstat` oder einem leichtgewichtigen Python-Query-Skript).
- **Datenausgabe:** Parsen der Query-Antworten in eine strukturierte Monitor-Ansicht, die den aktuellen Status (Offline, Startet, Initialisiert Hive, Online), Spielerzahl, Ping und Server-Version anzeigt.
- **Interaktives Menü:** Bereitstellung eines numerischen Eingabemenüs zur Ausführung der Kernlogik-Skripte (Start, Stop, Update, Backup).

3. Mod- und Key-Management (Synchronisations-Logik)

- **Konfigurations-Parser:** Ein Skript liest eine zentrale `mods.cfg` Zeile für Zeile ein.
- **Install/Update-Prozess:** Automatischer Aufbau des SteamCMD-Download-Befehls mit den entsprechenden Mod-IDs.
- **Bereinigungs-Routine:** Ein Abgleich der heruntergeladenen Mods im `/server_data/`-Verzeichnis mit der `mods.cfg`. Löschung von Ordnern, Symlinks und Keys, die in der Konfiguration nicht mehr existieren.

- **Key-Extraktion:** Ein rekursiver `find`-Befehl durchsucht alle installierten Mod-Ordner nach `.bikey`-Dateien und kopiert oder verlinkt diese automatisch flach in das Verzeichnis `/server_data/keys/`.

4. Start-Prozedur (State-Machine)

- **Inaktivitäts-Prüfung:** Das Skript bricht den Start ab, wenn `tmux ls` die definierte Serversitzung listet, ein DayZ-Prozess unter dem Benutzer existiert (`pgrep`) oder Netzwerkports für Game, Steam oder BattlEye blockiert sind (`ss` oder `netstat`).
- **Netzwerk-Sperre:** Setzen einer temporären `iptables`- oder `ufw`-Regel, die den Server-Port für externe Logins blockiert, um asynchrone Verbindungen während des Starts zu verhindern.
- **Ausführung:** Dynamische Generierung des Start-Strings inkl. `-mod=` Parametern. Start des Prozesses via `tmux new-session -d`.
- **Hive-Monitoring & Timeout:** Das Skript liest die aktuelle RPT-Logdatei in Echtzeit aus (`tail -f`). Erkennt der Parser den String "Hive loaded" (oder ähnlich), wird die Firewall-Regel gelöst. Wird der String nach einer festgelegten Zeit nicht erkannt, wird der Prozess gekillt und ein automatischer Neustart initiiert.

5. Stopp-Prozedur und Automatisches Backup

- **Status-Validierung:** Das Tool prüft, ob der Server gerade die Start-Routine durchläuft. Ist dies der Fall, wird der Stopp-Befehl verweigert, um False-Positives zu vermeiden.
- **Shutdown-Sequenz (via bercon-cli):**
  1. `say -1 "Server restart in X minutes!"` (verbunden mit einem Sleep-Timer).
  2. `#lock` (Sperrung des Servers für neue Verbindungen).
  3. `#kick` (Entfernen aller aktuell verbundenen Clients aus der Datenbank-Session).
  4. `#shutdown` (Sicheres Herunterfahren anweisen).
- **Prozess-Warten und Backup:** Eine Schleife prüft sekündlich den Wegfall der Prozess-ID. Sobald der Prozess beendet ist, wird sofort ein `tar`-Archiv des `storage_1`-Ordners (Hive) erstellt und mit einem Zeitstempel im Backup-Ordner abgelegt.
- **Fallback-Routine:** Bleibt die Prozess-ID nach einem Timeout (z. B. 5 Minuten nach dem `#shutdown`-Befehl) bestehen, erfolgt ein erzwingender `tmux kill-session` und ein anschließendes `kill -9 <PID>`.

6. Automatisierung und Auto-Recovery

- **Cron-like Loop:** Ein permanent als Systemd-Service oder `nohup`-Prozess laufender Hintergrund-Loop gleicht minütlich die Systemzeit mit den in einer `schedule.cfg` definierten Restart-Zeiten ab.
- **3-Stufen-Recovery:**
  - *Stufe 1 (Soft Restart):* Das Überwachungsskript meldet keine RCON-Verbindung mehr → Auslösen der regulären Stopp- und Start-Prozedur.
  - *Stufe 2 (Hard Reboot):* Stufe 1 schlägt fehl oder Ports hängen als "Zombie" fest → Harter Kill aller Benutzerprozesse.
  - *Stufe 3 (Rollback):* Das Skript erkennt im RPT-Log einen kritischen Datenbankfehler (Corrupted Data) → Das Server-Verzeichnis wird gesperrt, der aktuelle `storage_1` Ordner umbenannt, das letzte Backup-Archiv entpackt und der Server neu gestartet.